МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования
«Московский авиационный институт (национальный исследовательский университет)»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ
№ 3

Дисциплина:

Операционные системы и архитектура компьютера

Тема:

Управление потоками в ОС. Синхронизация потоков.

Выполнил:

студент группы М8О-205БВ-25

Герасимов А.С.

Москва, 2026 г.

1. Цель работы

Приобретение практических навыков в управлении потоками в операционной системе Linux с использованием библиотеки pthread, а также в обеспечении синхронизации между потоками с помощью семафоров.

2. Краткое описание архитектуры решения

Решение представляет собой программу на языке C, реализующую параллельный алгоритм быстрой сортировки (Quicksort).

Взаимодействие построено на порождении дочерних потоков. Главный поток выполняет разбиение массива (partition) и создает новый поток для сортировки левой части, а правую часть сортирует самостоятельно.

Для синхронизации и ограничения количества одновременно работающих потоков используется именованный семафор POSIX (sem_t). Начальное значение семафора равно максимальному числу потоков (-t).

Перед созданием потока вызывается sem_trywait. Если семафор равен 0 (лимит исчерпан), поток не создается, и данные сортируются последовательно. Когда дочерний поток завершает работу, он вызывает sem_post, освобождая слот для новых задач.

2.1 Схема взаимодействия компонентов

1. Главный поток читает аргументы (-t, -s), инициализирует семафор и массив.
2. Вызывается quicksort_parallel.
3. Внутри происходит вызов partition.
4. sem_trywait проверяет семафор (если мест нет — уход в quicksort_sequential).
5. pthread_create создает поток, который выполняет thread_func.
6. Главный поток сортирует правую часть.
7. pthread_join ожидает завершения дочернего потока.

3. Листинг ключевых фрагментов кода

3.1 Функция потока и логика создания

Функция thread_func выполняется в дочернем потоке. Она рекурсивно вызывает параллельную сортировку для своего куска, а затем освобождает семафор (сообщая, что слот свободен) и освобождает память.

```
static void* thread_func(void* arg) {
	ThreadArg* targ = (ThreadArg*)arg;
	quicksort_parallel(targ->arr,targ->left, targ->right, targ->depth);
```

```
    free(targ);
    sem_post(&sem);
    return NULL;
}
```

3.2 Синхронизация через семафор

Внутри функции quicksort_parallel проверяется возможность создания потока. Если sem_trywait возвращает 0, поток создается. В противном случае (если лимит достигнут) сортировка выполняется последовательно.

```
    if (sem_trywait(&sem) == 0) {
        ThreadArg *arg = malloc(sizeof(ThreadArg));
        arg->arr = arr;
        arg->left = left;
        arg->right = j;
        arg->depth = depth - 1;
        pthread_t tid;
        pthread_create(&tid, NULL, thread_func, arg);
        quicksort_parallel(arr, i, right, depth - 1);
        pthread_join(tid, NULL);
    } else {
        quicksort_sequential(arr, left, j);
        quicksort_sequential(arr, i, right);
    }
```

4. Результаты работы

4.1 Консольный вывод или логи

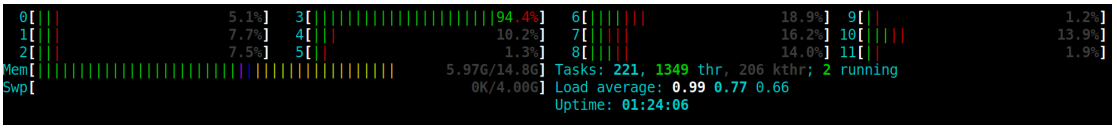
```
art@honor:~/projects/sem3/os/lab3/build$ ./lab3 -t 8 -s 50000000
Потоков: 8, Размер массива: 50000000
Отсортировано: да
Создано потоков: 53
Время: 1.5870 сек
```

4.2 Скриншоты результатов

Утилита ps -T показывает, что программа имеет несколько потоков. Все они имеют одинаковый PID, но разные SPID (TID).

```
art@honor:~$ ps aux | grep lab3
art        22324 99.7 25.1 3908948 3908100 pts/0 R+   11:30   0:06 ./lab3 -t 12 -s 1000000000
art        22369  0.0  0.0   6584  2468 pts/4    S+   11:30   0:00 grep --color=auto lab3
art@honor:~$ ps -T -p 22324
    PID    SPID TTY          TIME CMD
  22324   22324 pts/0    00:00:13 lab3
  22324   22374 pts/0    00:00:03 lab3
  22324   22375 pts/0    00:00:01 lab3
  22324   22376 pts/0    00:00:01 lab3
art@honor:~$
```

Загрузка ядер в программе htop



4.3 Результаты замеров (исследование ускорения)

Были проведены замеры времени выполнения для массива размером 50 000 000 элементов (порог переключения на последовательную сортировку — 5000 элементов).

Количество потоков (N)	Время выполнения (T, сек)	Ускорение (S = T1 / Tn)	Эффективность (E = S / N)
1	3.5159	1.00	1.00
2	3.4940	1.01	0.51
4	3.5420	0.99	0.25
8	1.6509	2.13	0.27
12	1.6808	2.09	0.17

Анализ результатов: Ускорение при переходе от 1 к 4 потокам практически отсутствует, что объясняется узким местом в пропускной способности шины памяти и конкуренцией за кэш (memory-bound). Резкий скачок ускорения (в 2.13 раза) происходит при использовании 8 потоков, когда массив дробится на достаточно мелкие части, помещающиеся в кэш L1/L2 ядер. Дальнейший рост потоков до 12 не дает прироста, так как физических ядер в процессоре всего 6, и в работу вступает технология Hyper-Threading (SMT). Также проявляется закон Амдала.

5. Выводы

В ходе работы были изучены механизмы создания потоков (pthread_create), их синхронизации с помощью семафоров (sem_t) и механизмы ожидания (pthread_join). Реализован параллельный алгоритм быстрой сортировки. Экспериментально доказано, что максимальное ускорение ограничено количеством физических ядер процессора, а также зависит от характера доступа к памяти и накладных расходов на синхронизацию.